

Classificação de Estrelas Através de sua Espectroscopia

Fernando Campos Silva Dal Maria*

Julia Veloso Dias*

fernandocsdm@gmail.com

julia.dias@sga.pucminas.br

Instituto de Ciências Exatas e Informática (ICEI)

Belo Horizonte, Minas Gerais, Brasil

ABSTRACT

Na astronomia os modelos de aprendizagem estão sendo largamente empregados. Na análise da espectroscopia muitas aplicações para a aprendizagem de máquina são encontradas. Outra área na qual a inteligência artificial tem sido empregada é na astronomia. Modelos de *Machine Learning*, por exemplo, vem sendo utilizados para classificação de galáxias, descrição de imagens, reconhecimento de padrões, identificação de peculiaridades de corpos celestes, anomalias e/ou detecção de objetos [12, 17]. Visto isso realizamos um estudo que realiza a aplicação da inteligência artificial para a classificação de estrelas baseado na análise espectroscópica do objeto astronômico. Nosso método faz uso da classificação estelar Morgan-Keenan, das magnitudes absolutas e aparentes das estrelas assim como da trigonometria de paralax para realizar uma classificação do objeto. Utilizamos os modelos *Random Forest* e *Bagging* para o aprendizado obtendo acurácia de 79% em ambos os modelos, com precisão de 37% para classificar estrelas anãs e 98% para classificar estrelas gigantes.

CCS CONCEPTS

• **Computer systems organization** → **Embedded systems**; *Redundancy*; Robotics; • **Networks** → Network reliability.

KEYWORDS

base de dados, modelos de classificação, *Random Forest*, *Bagging*, classificação

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Seminário 1, Set 2023, Belo Horizonte, MG, Brasil

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

ACM Reference Format:

Fernando Campos Silva Dal Maria and Julia Veloso Dias. 2026. Classificação de Estrelas Através de sua Espectroscopia. In *Proceedings of PUC Minas, Inteligência Artificial (Seminário 1)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUÇÃO

Os avanços tecnológicos e computacionais na última década contribuíram fortemente para o avanço dos modelos de inteligência artificial e para aplicação em diversas áreas, seja na saúde, na biologia, na geografia ou na física [14]. Outra área na qual a inteligência artificial tem sido empregada é na astronomia. Modelos de *Machine Learning*, por exemplo, vem sendo utilizados para classificação de galáxias, descrição de imagens, reconhecimento de padrões, identificação de peculiaridades de corpos celestes, anomalias e/ou detecção de objetos [12, 17]. Visto isso propomos um estudo que realiza a aplicação da inteligência artificial para a classificação de estrelas baseado na análise espectroscópica do objeto.

A Espectroscopia e sua Aplicação

A espectroscopia é um ponto central da astronomia. A análise das cores e do espectro luminoso revelam informações sobre a temperatura, sobre os átomos e sobre a velocidade do objeto astronômico observado. Essas informações são extremamente úteis para a análise da distância e da composição das estrelas [5, 17]. A classificação das estrelas, por sua vez, pode ser realizada através da espectroscopia. O sistema para classificação estelar Morgan-Keenan (MK) expandido estabelece um sistema de classificação preciso relacionado com a temperatura e coloração do objeto astronômico. Vale ressaltar, que o sistema MK utiliza a luminosidade estelar para realizar tal classificação [11, 17]. Veja a seguir o processo de classificação utilizado para o sistema MK: (1) é determinado um tipo espectral aproximado, (2) é determinada a classe de luminosidade, (3) por comparação com estrelas de luminosidade semelhante, é encontrado um tipo espectral preciso.

As classes O, B, A, F, G, K, M, L, T e Y representam estrelas com temperaturas diversas, enquanto os algarismos romanos I, II, III, IV e V são utilizados para representar as classes luminosidade [11, 19]. Classes de luminosidade

adicionais para subanãs (VI) e anãs brancas (VII) foram introduzidas posteriormente ao sistema MK [9]. Para uma descrição mais aprofundada com das classes de luminosidade estelar observe a Tabela 1.

Table 1: Descrição da classificação para luminosidade estelar, adaptado de Giridhar 2010 [9]

Luminosidade	Descrição
Ia	supergigantes mais luminosas
Ib	supergigantes menos luminosas
II	gigantes luminosas
III	gigantes normais
IV	subgigantes
V	sequência principal de estrelas
VI	subanãs
VII	anãs brancas

Para este trabalho inicial utilizamos a classificação luminosa apresentada nesta seção para classificar as estrelas como anãs (*'Dwarfs'*) ou gigantes (*'Giants'*).

2 REFERENCIAL TEÓRICO

Nesta seção serão contemplados os principais conceitos envolvidos neste artigo, assim como conceitos fundamentais para o entendimento das etapas de pré-processamento e de avaliação dos modelos aplicados.

Magnitude Absoluta e Paralax Trigonométrico

A magnitude absoluta de uma estrela, M_v é a magnitude aparente que uma estrela teria se fosse colocada a uma distância fixa da Terra, 10 parsecs. Para realizar uma estimativa da magnitude absoluta de uma estrela pode-se utilizar a seguinte relação entre distância e magnitude aparente:

$$(1) \quad M_v - m_v = -5 \log(d) + 5$$

na qual M_v é a magnitude absoluta, m_v é a magnitude aparente e d é a distância da estrela com relação a Terra [1, 5, 16].

Dessa forma, o paralax trigonométrico pode ser utilizado para obter uma estimativa da distância d e a relação apresentada permite obter, por consequência, a magnitude absoluta da estrela [1]. Neste artigo utilizamos ambos os conceitos para melhorar a performance do modelo aplicado a base de dados.

Algoritmos Ensemble Utilizados

Os métodos *Bagging*, *Random Forest* e *Boosting* são exemplos de métodos *ensemble*. O objetivo dessa técnica é criar um conjunto de classificadores para responder a um determinado problema de classificação de modo que seu desempenho seja melhor do que aquele apresentado por um modelo de classificação isolado. Ao combinar uma série de k modelos

os algoritmos *ensemble* realizam uma classificação do objeto baseando-se no voto majoritário dos modelos. Portanto, para o funcionamento desses algoritmos, um determinado conjunto de dados, D , é usado para criar k conjuntos de treinamento, $D_1, D_2, \dots, D_k$, onde D_i é usado para gerar o classificador M_i . No momento de realizar uma predição cada classificador M_i realiza sua previsão e o conjunto retorna um resultado com base nas decisões tomadas por esses classificadores [10, 14].

Esses conjuntos tendem a ser mais precisos quando comparados com os classificadores básicos, visto que, por mais que os classificadores M possam apresentar erros, o voto majoritário de k classificadores é capaz de compensar esses erros [10, 14].

Bagging. Utilizando a amostragem por reposição *Bootstrap*, o modelo *Bagging* realiza a geração de k conjuntos de treinamento distintos: $D_1, D_2, \dots, D_k$. Cada conjunto D_i é utilizado para o treinamento de um modelo M_i , de forma análoga ao descrito em *Métodos Ensemble*. Esse algoritmo permite o com que distintos modelos possam ser agrupados para realizar a classificação, ou seja, diferentemente do *Random Forest*, o modelo *Bagging* permite um agrupamento de modelos diferentes de árvores de decisão. Para realizar uma predição, k hipóteses são geradas pelo *Bagging* ao se aplicar uma nova entrada a cada um dos M_i modelos aprendidos. Com base nessas hipóteses o algoritmo irá selecionar aquela majoritária como a resposta adequada para classificar a entrada. Vale ressaltar que em caso de previsões numéricas contínuas (regressão) o resultado final é a média das k hipóteses obtidas [10, 14, 18]:

$$(2) \quad h(x) = \frac{1}{k} \sum_{i=1}^k h_i(x)$$

Random Forest. O *Random Forest* é um tipo de algoritmo *ensemble* que realiza o empacotamento de um conjunto de árvores de decisão CART. Esse algoritmo permite a geração de uma floresta de árvores de decisão diversificadas que realizam previsões para as entradas do modelo e tem seus resultados contabilizados para gerar uma predição mais precisa [10, 14, 18].

Para realizar a seleção de k conjuntos de dados, novamente, o método de amostragem *Bootstrap* é utilizado. Desse modo k conjuntos de dados são utilizados para a construção das árvores. Outro detalhe importante desse algoritmo é sua capacidade de selecionar atributos distintos para serem utilizados no aprendizado dos distintos modelos. Logo se houver n atributos, uma escolha comum é que cada divisão escolha aleatoriamente $\sqrt{n}$ ou $\log_2 n + 1$ atributos a serem considerados para problemas de classificação ou regressão. Uma melhoria adicional é usar a aleatoriedade na seleção do

valor do ponto de divisão: para cada atributo selecionado, amostramos aleatoriamente vários valores candidatos a partir de uma distribuição uniforme ao longo do intervalo do atributo [10, 14].

XGBoost. O *XGBoost*, originalmente nomeado *Extreme Gradient Boosting*, adota a abordagem de *Boosting*, construindo modelos sequencialmente para corrigir os erros do modelo anterior e já o termo *Gradient*, refere-se à otimização da função de perda usando gradientes. Essencialmente, o conjunto final permite que cada hipótese seja votada levando em consideração seus pesos e seu desempenho, as hipóteses que tiveram melhor desempenho em seus respectivos conjuntos de treinamento recebem mais peso de votação. Para regressão ou classificação binária, se utiliza[18]:

$$h(x) = \frac{1}{k} \sum_{i=1}^k z_i h_i(x)$$

Os pesos iniciam com $w_j = 1$ para todas as tuplas. A partir de um conjunto de treinamento amostrado um modelo M_1 é gerado e testado para identificar seus erros de predileção. Em seguida, para que o modelo M_2 tenha mais atenção aos erros de M_1 os pesos das tuplas de treinamento que foram erroneamente classificadas são aumentados. Esse processo continua até que os k modelos tenham sido treinados[18].

Métricas para a Avaliação dos Algoritmos de Classificação

Para a avaliação dos algoritmos de classificação, as métricas proporcionadas pela matriz de confusão serão utilizadas. A matriz de confusão proporciona os seguintes valores: verdadeiro positivo (VP), verdadeiro negativo (VN), falso positivo (FP) e falso negativo (FN), assim como suas taxas. A acurácia, a sensibilidade (*recall*), a precisão e o *F1-score* também podem ser obtidos. Tais métricas são utilizadas para avaliar modelos de classificação e estão detalhadas na Tabela 2.

Algoritmo SVM

Support Vector Machines (SVMs) são modelos de aprendizagem supervisionada, que utilizam de uma estratégia distinta daquela apresentada para os modelos *ensemble*. SVMs são muito utilizadas para solucionar problemas de classificação, sendo utilizadas tanto para dados lineares, quanto para não lineares. Elas usam um mapeamento não linear para transformar os dados de treinamento originais em espaços vetoriais em R^n [22]. Dentro desse espaço, o algoritmo busca identificar o hiperplano de separação linear ideal, isto é, um "limiar" que distingue os conjuntos de dados pertencentes a uma classe daqueles pertencentes a outra classe. Uma transformação não linear permite que, para um espaço em R^n , o hiperplano seja capaz de separar os objetos da base de

acordo com seus rótulos. Tais hiperplanos são determinados pelas SVMs com a utilização dos chamados vetores de suporte, conjuntos de treinamento considerados "essenciais" e margens definidas pelos vetores de suporte [6, 10]. Apesar das inúmeras vantagens do SVM, como sua habilidade de realizar generalizações, ele igualmente exibe algumas limitações. Essas desvantagens compreendem a complexidade envolvida na escolha dos parâmetros, a influência da complexidade do algoritmo no tempo de treinamento em conjuntos de dados extensos, as dificuldades associadas à criação de classificadores ideais para problemas multiclasse e as questões relacionadas ao desempenho do SVM em conjuntos de dados desequilibrados [6].

Algoritmo Multilayer Perceptron

Um *Multilayer Perceptron* (MLP), é uma rede neural artificial composta por unidades interconectadas (neurônios) dispostas em camadas. Um MLP é caracterizado por ter pelo menos uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída. A camada de entrada apresenta os valores das instâncias presentes na base de dados. Os neurônios das camadas ocultas, assim como os neurônios da camada de saída são interconectados sequencialmente, de modo que a camada de entrada se liga à primeira camada oculta e cada camada oculta C_i se conecte a próxima camada C_{i+1} , até que a última camada oculta se conecte a camada de saída [15].

Para realizar o processamento das n entradas o modelo MLP realiza, para cada *unidade_j*, um produto escalar entre um vetor de pesos $W \in R^n$ e o vetor ϕ que contém os valores das entradas conectadas a essa unidade. Outro escalar b denominado *bias* também é adicionado à soma para permitir com que a rede aprenda padrões mesmo quando todas as entradas são zero. Esse processo é apresentado em detalhes na Equação 4 [10, 15].

$$(4) \quad b + \sum_{i=1}^n w_i * \phi_i$$

O resultado desse produto escalar é utilizado por uma função de ativação $f(x)$ que retorna a saída do j -ésimo neurônio (O_j). Essa saída será utilizada como entrada para os neurônios da próxima camada oculta ou, caso não exista outra camada a frente, como a saída da rede neural [10, 15, 18].

O treinamento de um MLP envolve a otimização dos pesos das conexões para minimizar a diferença entre as previsões da rede e os valores reais do conjunto de treinamento. Isso é geralmente feito usando algoritmos de retropropagação (*backpropagation*), que ajustam iterativamente os pesos com base no gradiente da função de erro em relação aos pesos. A Equação 5 representa como o erro é calculado para a camada

Table 2: Equações e descrições das métricas para avaliação de modelos de classificação. Adaptado de [4, 21]

Métrica	Descrição	Equação
Acurácia	é a proporção de predições corretas feitas pelo modelo em relação ao número total de predições	$\frac{VP+VN}{VP+FN+FP+VN}$
<i>Recall</i>	é a porcentagem de casos positivos corretamente classificados como pertencentes a classe positiva dentre todos aqueles que deveriam ser classificados como pertencentes a classe positiva	$\frac{VP}{VP+FN}$
Precisão	é a proporção de exemplos classificados como positivos que são realmente positivos, em relação ao número total de exemplos classificados como positivos	$\frac{VP}{VP+FP}$
F1-score	é a média armônica entre a precisão e o <i>recall</i>	$\frac{2*precisao*recall}{precisao+recall}$
Taxa de VP	é a porcentagem de casos positivos corretamente classificados como pertencentes à classe positiva	$\frac{VP}{VP+FN}$
Taxa de FN	é a porcentagem de casos positivos classificados erroneamente como pertencentes à classe negativa	$\frac{FN}{VP+FN}$
Taxa de VN	é a porcentagem de casos negativos classificados corretamente como pertencentes à classe negativa	$\frac{VN}{VN+FP}$
Taxa de FP	é a porcentagem de casos negativos classificados erroneamente como pertencentes à classe positiva	$\frac{FP}{VN+FP}$

de saída, enquanto a Equação 6 apresenta como o erro é calculado para as demais camadas ocultas da rede neural [10, 15, 18].

$$(5) \quad \delta_j = O_j(1 - O_j)(O_j - T_j)$$

$$(6) \quad \delta_j = O_j(1 - O_j)\left(\sum_l \delta_l * w_{jl}\right)$$

onde δ_j representa o erro do neurônio, T_j representa a saída real (o rótulo da instância) e $\sum_l \delta_l * w_{jl}$ representa a soma ponderada dos erros das l unidades conectadas à unidade j na próxima camada. Finalmente observe nas Equações 7 e 8 o cálculo utilizado para realizar o ajuste dos pesos de cada neurônio [10, 15, 18].

$$(7) \quad \Delta w_{ij} = \eta \delta_j \phi_i$$

$$(8) \quad w_{ij} = w_{ij} - \Delta w_{ij}$$

onde η representa a taxa de aprendizado e w_{ij} o peso utilizado entre a camada i e a camada j [10, 15].

Algoritmo K-Nearest-Neighbours

O algoritmo *K-Nearest-Neighbours* (KNN) é um algoritmo de classificação que se baseia na proximidade entre exemplos para realizar a classificação de novos. Dado um exemplo de teste, o algoritmo encontra os k exemplos de treinamento mais próximos, utilizando uma métrica de distância para calcular a proximidade entre eles. A classe mais comum entre esses k vizinhos é selecionada [4, 10]. Uma das vantagens do KNN é que ele não faz suposições sobre a distribuição dos dados, o que o torna útil em problemas em que a distribuição dos dados é desconhecida. Além disso, ele pode ser facilmente adaptado para lidar com dados de diferentes tipos, como dados numéricos e categóricos. No entanto, o KNN pode ser sensível a dados ruidosos ou desbalanceados, o que pode afetar sua precisão [4].

3 DESCRIÇÃO DA BASE DE DADOS

A base de dados foi selecionada pela plataforma do "Strasbourg astronomical Data Center". Seus dados foram obtidos através da ferramenta Vizier, também pertencente ao Centro de Dados Astronômicos de Strasbourg, exceto o atributo "Amag" que foi calculado com a relação entre parallax

trigonométrico e magnitude absoluta previamente apresentada. Essa base utiliza dados espectrais para classificar os objetos astronômicos conforme o sistema de MK. Para um maior detalhamento dos atributos da base veja a Tabela 3 com a descrição dos atributos e seus tipos e a Tabela 4 com as estatísticas dos atributos contínuos presentes na base de dados antes de realizar o pré-processamento e da criação do atributo "Amag".

Table 3: Descrição dos atributos presentes na base de dados.

Atributo	Descrição	Valores
ID	Identificador do registro	inteiros 64 bits
Vmag	Magnitude aparente da estrela	reais 64 bits
Amag	Magnitude absoluta da estrela	reais 64 bits
Plx	Paralax trigonométrico em mili-segundos de arco	reais 64 bits
e_Plx	Erro relativo do paralax trigonométrico	reais 64 bits
B-V	Índice de coloração Johnson B-V	reais 64 bits
SpType	Classificação das estrelas	Conforme o sistema MK

Para o teste inicial do tratamento de dados e do modelo, um atributo adicional foi gerado para ser utilizado como a classe de classificação. O modelo de aprendizado supervisionado utilizará tal rótulo, mais detalhes sobre o modelo serão apresentados adiante neste projeto. O novo atributo adicionado a base tem o nome de "Target" e apresenta apenas dois valores possíveis: Anãs ('Dwarfs') e Gigantes ('Giants'). Sua geração foi realizada através da interpretação do atributo "SpType" conforme o sistema de MK descrito na introdução. A geração do atributo pode ser encontrada em: Geração do atributo de classificação.

4 ETAPAS DE PRÉ-PROCESSAMENTO

Para a base em questão foram realizadas as seguintes etapas iniciais de pré-processamento nessa ordem:

- (1) Importação dos dados do Centro de Dados Astronômicos de Strasbourg;
- (2) Remoção dos registros com valores nulos (faltantes) para o atributo "SpType";

- (3) Geração da classe alvo da predição;
- (4) Remoção de atributos inadequados para o treinamento e teste do modelo ("ID" e "SpType");
- (5) Remoção de *outliers*
- (6) Tratamento dos valores faltantes para os demais atributos da base;
- (7) Criação do atributo "Amag" com o uso dos atributos "Plx" e "Vmag".
- (8) Normalização de dados

Etapas de 1 a 4

Para a etapa 1 aproximadamente 100000 registros foram carregados e não apresentaram duplicatas. Os registros carregados, segundo o provedor dos dados, também não tinham passado por nenhum pré-processamento. Desses registros, na etapa 2, foram excluídos aqueles que apresentaram valores faltantes para o atributo "SpType". Sua remoção se deve ao fato de que, como esse atributo será utilizado para a geração da classe alvo da predição, os registros que não apresentarem tal valor não puderam ser devidamente classificados segundo a metodologia adotada para este experimento.

Após essa manipulação inicial restaram 97377 registros para serem utilizados na etapa 3. Para este estudo a geração da classe alvo foi realizada através da classificação de luminosidade de cada estrela, devido à possibilidade de classificação o objeto astronômico como sendo uma estrela anã ou gigante (observe a Tabela 1). Sendo assim aquelas estrelas que apresentaram classificação de luminosidade I, II ou III foram classificadas como gigantes, enquanto aquelas com classificação superior a III foram classificadas como anãs. Uma observação importante é que nem todas as estrelas apresentam a classificação de luminosidade, dessa forma, para este estudo inicial, aqueles registros que não apresentavam essa classificação foram excluídos.

Finalmente nos atentamos a etapa 4, na qual foram removidos os atributos "ID" e "SpType" dos 47845 registros restantes. O atributo "ID" foi removido por sua falta de correlação com as características da estrela, enquanto o atributo "SpType" foi removido devido a sua alta correlação com os rótulos gerados para o aprendizado.

Remoção de outliers

Para tornar a base de dados mais homogênea os dados discrepantes (*outliers*) presentes na amostra foram removidos. Sua remoção permite que o modelo de aprendizagem aplicado tenha um desempenho melhor ao avaliar a amostra e prever os resultados.

Para realizar a remoção dos *outliers* uma estratégia baseada na análise do intervalo interquartil (IQR) foi adotada [23]. Inicialmente os quartis 0.25 e 0.75 da amostra foram calculados para cada atributo (Q1 e Q3). Os quartis obtidos foram

Table 4: Estatísticas dos atributos contínuos presentes na base de dados inicial.

Atributo	Média	Desvio Padrão	Min	25%	50%	75%	Max
Vmag	8.369723	1.313881	-1.44	7.640	8.440	9.140	14.08
Plx	7.212444	11.349038	-54.95	2.510	4.630	8.410	772.33
e_Plx	1.365388	1.816845	0.38	0.880	1.100	1.390	114.46
B-V	0.704729	0.489686	-0.40	0.348	0.612	1.075	5.46

utilizados para obter o IQR através da relação:

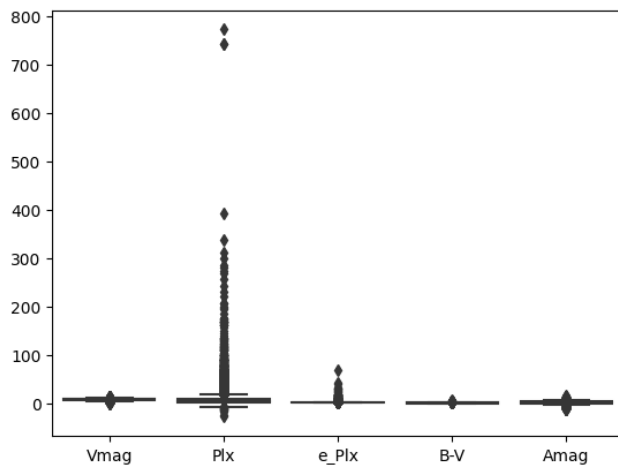
$$(3) \quad IQR = Q3 - Q1$$

o intervalo obtido contempla 50% dos dados presentes em cada atributo. Em seguida as relações:

$$(4) \quad \text{limite_superior} = Q3 + 1.5 * IQR$$

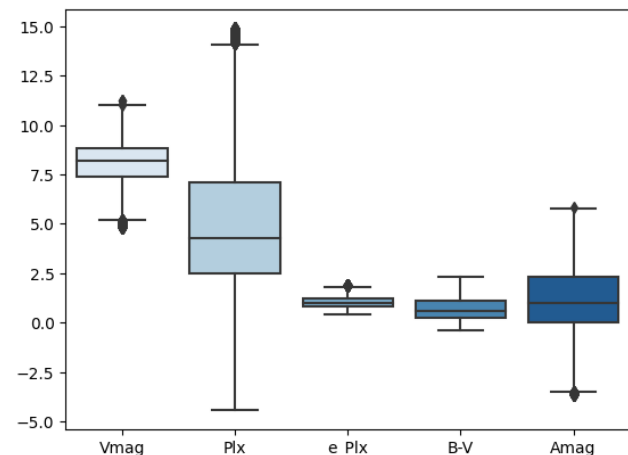
$$(5) \quad \text{limite_inferior} = Q1 - 1.5 * IQR$$

foram utilizadas para obter os limites superiores e inferiores para os valores de cada atributo. Dessa forma o seguinte gráfico da Figura 1 foi obtido.

Figure 1: Boxplot apresentando o conjunto de dados presentes na amostra não tratada.

Ao analisar o *boxplot* da Figura 1 é possível perceber a notória presença dos outliers para o atributo "Plx", assim como a presença de diversos outros valores discrepantes nos demais atributos. Para mitigar esse problema um corte da amostra foi realizado considerando os limites superiores e inferiores de cada atributo. Dessa forma os registros contendo valores que superaram os limites superiores ou inferiores de seus atributos foram removidos. Com o corte foram removidos 8725 registros e outros 39051 seguiram para as próximas

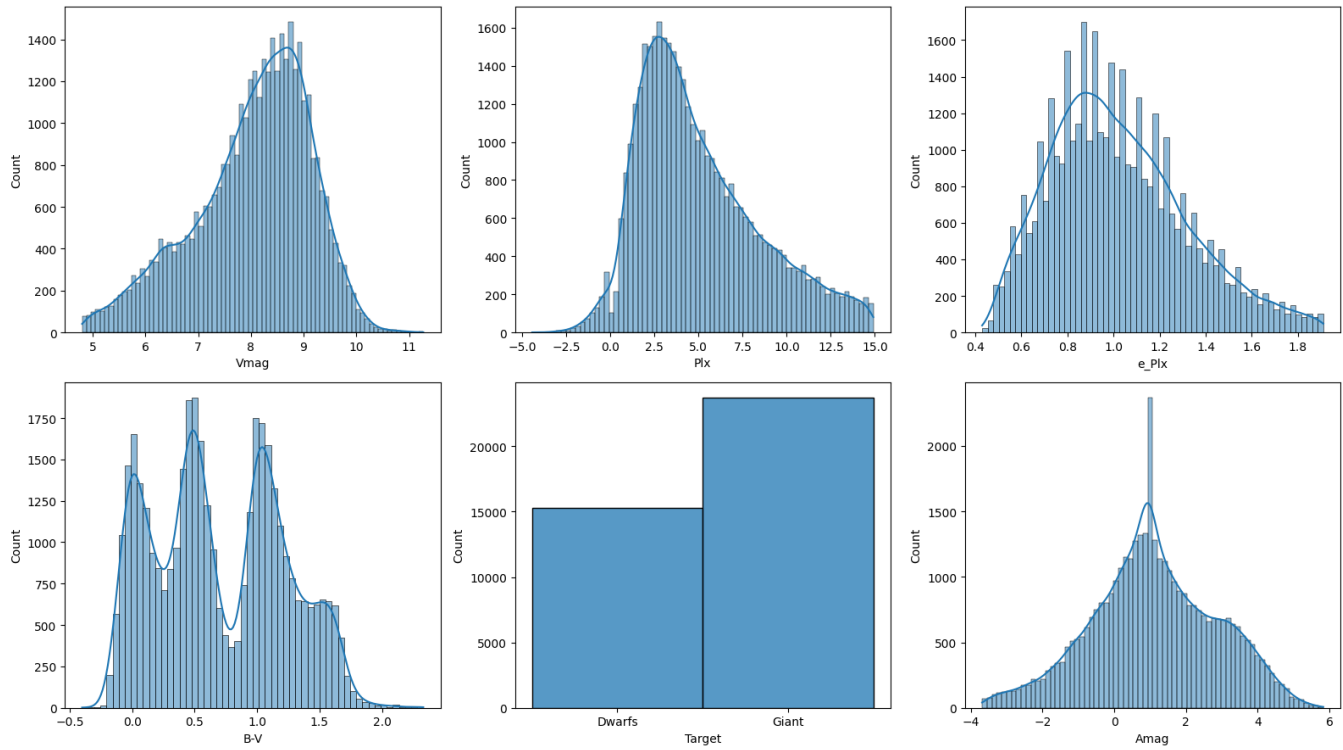
etapas do pré-processamento. Observe nas Figuras 2 e 3 as distribuições obtidas para cada um dos atributos da amostra após a remoção dos *outliers*.

Figure 2: Boxplot apresentando o conjunto de dados presentes na amostra tratada.

Etapas 6 e 8

O tratamento dos valores faltantes na etapa 6 foi realizado utilizando o algoritmo k-NN (*k-Nearest Neighbors*), um algoritmo de classificação que se baseia na proximidade entre exemplos para realizar a classificação de novos. Dado um exemplo de teste, o algoritmo encontra os *k* exemplos de treinamento mais próximos, utilizando uma métrica de distância para calcular a proximidade entre eles. A classe mais comum entre esses *k* vizinhos é selecionada [2, 4, 10]. Uma das vantagens do k-NN é que ele não faz suposições sobre a distribuição dos dados, o que o torna útil em problemas em que a distribuição dos dados é desconhecida. Além disso, ele pode ser facilmente adaptado para lidar com dados de diferentes tipos, como dados numéricos e categóricos. No entanto, o k-NN pode ser sensível a dados ruidosos ou desbalanceados, o que pode afetar sua precisão [4]. Sendo assim o uso

Figure 3: Distribuição dos dados por atributo após a remoção de outliers



da classe *KNNInputer* da biblioteca *scikit-learn* do python foi utilizada para realizar o preenchimento dos atributos faltantes.

O atributo "*Amag*" foi criado com o uso da Equação 1 apresentada na introdução. Logo os atributos "*Plx*" e "*Vmag*" foram utilizados nos cálculos para a obtenção da magnitude absoluta. Uma remoção adicional de 16 registros foi realizada, visto que esses registros apresentavam o atributo "*Plx*" igual a 0.0, o que inviabilizaria a divisão necessária para se obter a distância da estrela com relação a terra (veja mais em: Tratamento inicial dos dados). O resultado final do tratamento inicial conta com 8741 registros removidos e 39035 registros preparados para as próximas etapa.

Para finalizar, uma etapa de normalização de dados foi adicionada para melhorar o desempenho do modelo de SVM. Essa etapa faz com que seja possível comparar variáveis de diferentes unidades e magnitudes com facilidade, permitindo interpretações e comparações mais claras, além de promover melhores resultados para modelos como a SVM [13, 20]. Uma pesquisa realizada por Dalwinder Singh e Birmohan Singh avaliou sete diferentes métodos de normalização e os resultados indicaram que os métodos Min-Max (que normaliza os dados entre o intervalo [0, 1]) e z-score apresentaram

um desempenho superior em comparação com outras técnicas de normalização tanto para modelos SVM quanto para alguns outros modelos [20]. Visto isso, selecionamos o modelo Min-Max para realizar a normalização dos dados neste artigo. Observe a Equação 6 que representa o processo de normalização realizado pelo Min-Max:

$$(6) \quad x'_{i,n} = \frac{x_{i,n} - \min(x_i)}{\max(x_i) - \min(x_i)} (nMax - nMin) + nMin$$

Onde *min* e *max* representam os valores mínimo e máximo para o *i*-ésimo conjunto de valores. Os limites para a reescala dos dados são indicados como *nMin* para o limite inferior e *nMax* para o limite superior. Neste estudo, apenas a escala [0, 1] será utilizada (Equação 6 foi retirada do artigo de Dalwinder Singh e Birmohan Singh) [10, 20].

Balanceamento de dados

Para este estudo tanto técnicas de *under-sampling*, quanto de *over-sampling* foram testadas para balancear o atributo "*Target*" da classificação. A técnica de *under-sampling* realiza um corte nos dados com o objetivo de reduzir o número de instâncias da classe majoritária. Enquanto a técnica de *over-sampling* cria novas instâncias pertencentes a classe

minoritária, até que o número de instâncias pertencentes a cada classe sejam iguais [4].

A técnica preferida nos testes de *over-sampling* foi a Sobreamostragem Minoritária Sintética (SMOTE) que realiza a criação de instâncias sintéticas obtidas através de segmentos de reta que conectam amostra da classe minoritária aos k vizinhos mais próximos dentro da mesma classe [7]. Para essa técnica de balanceamento um cuidado adicional foi tomado para realizar o balanceamento evitando o vazamento de dados, ou seja, o balanceamento foi realizado apenas após a separação dos dados em conjunto de treino e teste, sendo aplicado somente ao conjunto de dados de treinamento.

Com relação ao *under-sampling* a técnica preferida utiliza de heurísticas baseadas no algoritmo *k-Nearest Neighbors*, que será contemplado com mais detalhes adiante no texto. Para utilizar tais heurísticas o método *NearMiss* versão 1 da biblioteca *sci-kit learn* do python foi utilizado. Ele permite manter amostras da classe majoritária que são mais próximas de n amostras da classe minoritária, preservando assim informações importantes para a classificação [2, 3, 8].

Após a realização dos testes, selecionamos, para este artigo, apenas os resultados obtidos pelo método preferido para *under-sampling*. A preferência pelo método de *under-sampling* se deve a evitar a geração de instâncias fictícias, que podem levar o modelo a tomar conclusões falaciosas sobre os dados, já que suas conclusões serão tomadas a partir de um conjunto de treinamento com instâncias fictícias. Além disso é importante ressaltar que a base de dados utilizada apresenta grande quantidade de instâncias, logo a remoção de um conjunto de objetos da classe majoritária não causará grandes problemas com relação a quantidade de instâncias disponíveis para o treinamento. Aquelas instâncias removidas da classe majoritária foram armazenadas para serem utilizadas no teste dos modelos já treinados.

5 APLICAÇÃO DO MODELO AOS DADOS

Após a realização da divisão de dados em treino e teste uma validação cruzada aninhada foi utilizada para validar o conjunto de treino dos modelos, de modo que o conjunto de dados da base foi dividido inicialmente em treino e teste e posteriormente o treino foi redividido conforme especificado em 4 para uma amostragem com validação. Os resultados aqui apresentados representam o resultado final para cada modelo após a validação e utilizando todo o conjunto de treinamento com o conjunto de teste. Foi determinado o uso do *Random Forest* para o treinamento inicial, devido à abrangência da base e aos benefícios que esse algoritmo de aprendizado de máquina oferece. Outro algoritmo *ensemble* utilizado foi o *XGBoost*, que é uma extensão do algoritmo de *Gradient Boosting*, no qual que combina várias árvores de decisão para formar um modelo mais robusto. Após o uso dos métodos *ensemble*, também foram utilizados os modelo

Support Vector Machine (SVM), *Multilayer Perceptron (MLP)* e *K-nearest-neighbours (KNN)* na tentativa de obter um resultado melhor na classificação das estrelas. Observe na seção de validação e comparação, disposta mais adiante neste artigo, as comparações e análises estatísticas relativas ao desempenho de cada um desses algoritmos.

Resultados do modelo *Random Forest*

Análise de resultados do modelo *Random Forest*

As métricas obtidas para a avaliação desse algoritmo são apresentadas na Tabela 5. A classe 0 representa as estrelas anãs e a classe 1 representa as estrelas gigantes. O modelo classificou incorretamente $304 + 1121 = 1425$ registros e classificou corretamente $2725 + 3486 = 6211$ registros, respectivamente.

A classificação dos objetos pertencentes a classe 0 foi satisfatória (Taxa de VP = 90% aproximadamente), entretanto a classificação daqueles objetos pertencentes a classe 1 não apresentou um desempenho semelhante (Taxa de VP = 75% aproximadamente). O modelo, em um patamar geral, obteve taxas relativamente semelhantes àquelas obtidas nos testes envolvendo o modelo *Bagging*.

Resultados das Métricas de Avaliação de Desempenho do modelo *Random Forest*

A métrica de avaliação de desempenho, no contexto da análise de modelos de aprendizado de máquina e estatística, é uma medida que possibilita a quantificação do grau de eficácia com o qual um modelo ou algoritmo desempenha uma tarefa específica. Dentre os métodos de avaliação de desempenho, foi calculado as seguintes métricas: precisão, *recall*, *F1-Score* e acurácia.

A precisão, é essa uma métrica de avaliação de desempenho que mensura a proporção de previsões corretas em relação ao total de previsões para cada classe. No caso da classe 0, foi obtido uma precisão de 71% das previsões corretas. Para a classe 1, a precisão é de 92% de previsões corretas, sendo possível concluir que a precisão é alta para a classe 1, indicando que quando o modelo prevê a classe 1, ele está correto na maioria das vezes. No entanto, a precisão para a classe 0 é relativamente baixa, sugerindo que há mais falsos positivos nessa classe.

A sensibilidade, ou *recall*, é outra métrica de avaliação que quantifica a proporção de exemplos positivos reais corretamente identificados pelo modelo em relação ao total de exemplos reais positivos. Para a classe 0, a sensibilidade é de 90% dos exemplos positivos capturados pelo modelo. Na classe 1, a sensibilidade é de 75% dos exemplos positivos corretamente identificados, sendo possível concluir a sensibilidade para a classe 0 é alta, indicando que o modelo captura a maioria dos exemplos reais da classe 0. No entanto, a sensibilidade para a classe 1 é um pouco menor, o

que significa que o modelo perde alguns exemplos reais da classe.

O *F1-Score* é uma métrica que combina precisão e sensibilidade em uma única medida, fornecendo uma média harmônica ponderada dessas duas métricas. Para a classe 0, o *F1-Score* é de 79%, enquanto para a classe 1, é de 83%. Quanto mais próximo de 1, melhor o desempenho do modelo. Concluindo, como o *Random Forest*, que o valor de 79% para a classe 0 indica um equilíbrio moderado entre precisão e sensibilidade, enquanto o valor de 83% para a classe 1 é alto, sugerindo um bom equilíbrio entre precisão e sensibilidade para essa classe.

Por fim, a acurácia é uma métrica que avalia a proporção de exemplos classificados corretamente em relação ao total de exemplos. No geral, o modelo alcança uma acurácia de 81%, o que indica uma capacidade geral de fazer previsões corretas de 81% em toda a classificação considerando um total de 7636 instâncias no conjunto de dados de teste, veja a Tabela 5 com os resultados.

Table 5: Métricas de Avaliação de Desempenho do modelo *Random Forest*.

Classe	Precisão	Recall	F1-Score	Suporte
anã (0)	0.71	0.90	0.79	3029
gigante (1)	0.91	0.76	0.83	4607
Acurácia	-	-	0.81	7636

Método *Bagging* para comparação

Análise de resultados do método *Bagging*

As métricas obtidas para a avaliação desse algoritmo são apresentadas na Tabela 6. A classe 0 representa as estrelas anãs e a classe 1 representa as estrelas gigantes. O modelo classificou incorretamente $473 + 950 = 1423$ registros e classificou corretamente $2556 + 3657 = 6213$ registros, respectivamente.

A classificação dos objetos pertencentes a classe 0 foi satisfatória (Taxa de VP = 84%), entretanto a classificação daqueles objetos pertencentes a classe 1 não apresentou um desempenho semelhante (Taxa de VP = 79%). O modelo, em um patamar geral, obteve taxas relativamente semelhantes às aquelas obtidas nos testes envolvendo o modelo *Bagging*.

Resultados das Métricas de Avaliação de Desempenho do método *Bagging*

Assim como no método de *Random Forest*, no *Bagging* a métrica de avaliação de desempenho, no contexto da análise de modelos de aprendizado de máquina e estatística, é uma medida que possibilita a quantificação do grau de eficácia com o qual um modelo ou algoritmo desempenha uma tarefa

específica. Dentre os métodos de avaliação de desempenho, foi calculado as seguintes métricas: precisão, *recall*, *F1-Score* e acurácia.

A precisão, é essa uma métrica de avaliação de desempenho que mensura a proporção de previsões corretas em relação ao total de previsões para cada classe. No caso da classe 0, foi obtido uma precisão de 73% das previsões corretas. Para a classe 1, a precisão é de 89% de previsões corretas, sendo possível concluir, que assim como o *Random Forest*, que a precisão é alta para a classe 1, indicando que quando o modelo prevê a classe 1, ele está correto na maioria das vezes. No entanto, a precisão para a classe 0 é relativamente baixa, sugerindo que há mais falsos positivos nessa classe.

A sensibilidade, ou *recall*, é outra métrica de avaliação que quantifica a proporção de exemplos positivos reais corretamente identificados pelo modelo em relação ao total de exemplos reais positivos. Para a classe 0, a sensibilidade é de 84% dos exemplos positivos foram capturados pelo modelo. Na classe 1, a sensibilidade é de 79% dos exemplos positivos corretamente identificados, sendo possível concluir, que assim como o *Random Forest*, a sensibilidade para a classe 0 é alto, indicando que o modelo captura a maioria dos exemplos reais da classe 0. No entanto, a sensibilidade para a classe 1 é um pouco menor, o que significa que o modelo perde alguns exemplos reais da classe.

O *F1-Score* é uma métrica que combina precisão e sensibilidade em uma única medida, fornecendo uma média harmônica ponderada dessas duas métricas. Para a classe 0, o *F1-Score* é de 78%, enquanto para a classe 1, é de 84%. Quanto mais próximo de 1, melhor o desempenho do modelo. Concluindo, como o *Random Forest*, que o valor de 78% para a classe 0 indica um equilíbrio moderado entre precisão e sensibilidade, enquanto o valor de 84% para a classe 1 é alto, sugerindo um bom equilíbrio entre precisão e sensibilidade para essa classe.

Por fim, a acurácia é uma métrica que avalia a proporção de exemplos classificados corretamente em relação ao total de exemplos. No geral, o modelo alcança uma acurácia de 0,81, o que indica uma capacidade geral de fazer previsões corretas de 81% em toda a classificação considerando um total de 7636 instâncias no conjunto de dados de teste, podendo assim concluir que a acurácia geral é consistente com a precisão da classe 1, veja a Tabela 6 com os resultados.

Resultados do modelo *XGBoost*

Análise de resultados do modelo *XGBoost*

O modelo *XGBoost* obteve um desempenho razoável, com 80% dos registros sendo classificados corretamente. As métricas obtidas para a avaliação desse algoritmo são apresentadas na Tabela 7. A classe 0 representa as estrelas anãs e a

Table 6: Métricas de Avaliação de Desempenho do modelo Bagging.

Classe	Precisão	Recall	F1-Score	Suporte
anã (0)	0.73	0.84	0.78	3029
gigante (1)	0.89	0.79	0.84	4607
Acurácia	-	-	0.81	7636

classe 1 representa as estrelas gigantes. O modelo classificou incorretamente $458 + 1077 = 1535$ registros e classificou corretamente $2571 + 3530 = 6101$ registros, respectivamente.

A classificação dos objetos pertencentes a classe 0 foi satisfatória (Taxa de VP = 85%), entretanto a classificação daqueles objetos pertencentes a classe 1 não apresentou um desempenho semelhante (Taxa de VP = 77%). O modelo, em um patamar geral, obteve taxas relativamente semelhantes àquelas obtidas nos testes envolvendo o modelo *Bagging*.

Resultados das Métricas de Avaliação de Desempenho do modelo XGBoost

A precisão avalia a proporção de previsões corretas em relação ao total de previsões para cada classe. No caso da classe 0, foi obtido uma precisão de 70% das previsões corretas. Para a classe 1, a precisão é de 89% de previsões corretas, concluindo que para a classe 1 é alta, indicando que quando o modelo prevê a classe 1, está correto 89% das vezes. No entanto, para a classe 0, a precisão é mais baixa, sugerindo um número significativo de falsos positivos.

A sensibilidade, ou *recall*, é outra métrica de avaliação que quantifica a proporção de exemplos positivos reais corretamente identificados pelo modelo em relação ao total de exemplos reais positivos. Para a classe 0, a sensibilidade é de 85% dos exemplos positivos capturados pelo modelo. Na classe 1, a sensibilidade é de 77% dos exemplos positivos foram corretamente identificados, concluindo que sensibilidade para a classe 0 é alta, significando que o modelo identifica corretamente 85% dos exemplos positivos reais da classe 0. No entanto, para a classe 1, a sensibilidade é razoável, indicando que apenas 77% dos exemplos positivos da classe 1 são capturados pelo modelo.

O *F1-Score* é uma métrica que combina precisão e sensibilidade em uma única medida, fornecendo uma média harmônica ponderada dessas duas métricas. Para a classe 0, o *F1-Score* é de 77%, enquanto para a classe 1, é de 82%, considerando que quanto mais próximo de 1, melhor o desempenho do modelo. O *F1-Score* é alto para a classe 1, refletindo um bom equilíbrio entre precisão e sensibilidade. Para a classe 0, o *F1-Score* é mais baixo, sugerindo um compromisso entre precisão e sensibilidade.

Por fim, a acurácia é uma métrica que avalia a proporção de exemplos classificados corretamente em relação ao total de exemplos. No geral, o modelo alcança uma acurácia de 0,80, o que indica uma capacidade geral de fazer previsões corretas de 80% em toda a classificação considerando um total de 7636 instâncias no conjunto de dados de teste, veja a Tabela 7 com os resultados.

Table 7: Métricas de Avaliação de Desempenho do modelo XGBoost

Classe	Precisão	Recall	F1-Score	Suporte
anã (0)	0.70	0.85	0.77	3029
gigante (1)	0.89	0.76	0.82	4607
Acurácia	-	-	0.80	7636

Resultados do modelo SVM

Análise de resultados do modelo SVM

O modelo SVM obteve um desempenho razoável, com 81% dos registros sendo classificados corretamente. As métricas obtidas para a avaliação desse algoritmo são apresentadas na Tabela 8. A classe 0 representa as estrelas anãs e a classe 1 representa as estrelas gigantes. O modelo classificou incorretamente $679 + 726 = 1.405$ registros e classificou corretamente $835 + 8.677 = 9.512$ registros, respectivamente.

A classificação dos objetos pertencentes a classe 1 foi satisfatória (Taxa de VP = 92% aproximadamente), entretanto a classificação daqueles objetos pertencentes a classe 0 não apresentou um desempenho semelhante (Taxa de VP = 70% aproximadamente).

Resultados das Métricas de Avaliação de Desempenho do método SVM

A precisão avalia a proporção de previsões corretas em relação ao total de previsões para cada classe. No caso da classe 0, foi obtido uma precisão de 0,70, o que equivale a 70% das previsões corretas. Para a classe 1, a precisão é de 0,93, representando 92% de previsões corretas, concluindo que para a classe 1 é alta, indicando que quando o modelo prevê a classe 1, está correto 92% das vezes. No entanto, para a classe 0, a precisão é mais baixa, sugerindo um número significativo de falsos positivos.

A sensibilidade, ou *recall*, é outra métrica de avaliação que quantifica a proporção de exemplos positivos reais corretamente identificados pelo modelo em relação ao total de exemplos reais positivos. Para a classe 0, a sensibilidade é de 0,91, ou seja, 91% dos exemplos positivos foram capturados pelo modelo. Na classe 1, a sensibilidade é de 0,74, indicando que 74% dos exemplos positivos foram corretamente

identificados, concluindo que sensibilidade para a classe 1 é razoável, significando que o modelo identifica corretamente 74% dos exemplos positivos reais da classe 1. No entanto, para a classe 0, a sensibilidade é alta, indicando que 91% dos exemplos positivos da classe 0 são capturados pelo modelo.

O *F1-Score* é uma métrica que combina precisão e sensibilidade em uma única medida, fornecendo uma média harmônica ponderada dessas duas métricas. Para a classe 0, o *F1-Score* é de 0,79, enquanto para a classe 1, é de 0,82, considerando que quanto mais próximo de 1, melhor o desempenho do modelo. O *F1-Score* é alto para a classe 1, refletindo um bom equilíbrio entre precisão e sensibilidade. Para a classe 0, o *F1-Score* é mais baixo, sugerindo um compromisso entre precisão e sensibilidade.

Por fim, a acurácia, é uma métrica que avalia a proporção de exemplos classificados corretamente em relação ao total de exemplos. No geral, o modelo alcança uma acurácia de 0,87, o que indica uma capacidade geral de fazer previsões corretas de 81% em toda a classificação considerando um total de 7.636 instâncias no conjunto de dados de teste, veja a Tabela 8 com os resultados.

Table 8: Métricas de Avaliação de Desempenho do modelo SVM.

Classe	Precisão	Recall	F1-Score	Suporte
anã (0)	0.70	0.91	0.79	3029
gigante (1)	0.92	0.74	0.82	4607
Acurácia	-	-	0.81	7.636

Resultados do modelo MLP

Análise de resultados do modelo *Multilayer Perceptron* (MLP)

As métricas obtidas para a avaliação desse algoritmo são apresentadas na Tabela 9. A classe 0 representa as estrelas anãs e a classe 1 representa as estrelas gigantes. O modelo classificou incorretamente $491 + 846 = 1.337$ registros e classificou corretamente $2.538 + 3.716 = 6.074$ registros, respectivamente.

A classificação dos objetos pertencentes a classe 1 foi satisfatória (Taxa de VP = 90% aproximadamente), entretanto a classificação daqueles objetos pertencentes a classe 0 não apresentou um desempenho semelhante (Taxa de VP = 73% aproximadamente).

Resultados das Métricas de Avaliação de Desempenho do método *Multilayer Perceptron*

A precisão avalia a proporção de previsões corretas em relação ao total de previsões para cada classe. No caso da

classe 0, foi obtido uma precisão de 0,73, o que equivale a 73% das previsões corretas. Para a classe 1, a precisão é de 0,90, representando 90% de previsões corretas, concluindo que para a classe 1 é alta, indicando que quando o modelo prevê a classe 1, está correto 90% das vezes. No entanto, para a classe 0, a precisão é mais baixa, sugerindo um número significativo de falsos positivos.

A sensibilidade, ou *recall*, é outra métrica de avaliação que quantifica a proporção de exemplos positivos reais corretamente identificados pelo modelo em relação ao total de exemplos reais positivos. Para a classe 0, a sensibilidade é de 0,87, ou seja, 87% dos exemplos positivos foram capturados pelo modelo. Na classe 1, a sensibilidade é de 0,79, indicando que 79% dos exemplos positivos foram corretamente identificados, concluindo que sensibilidade para a classe 1 é razoável, significando que o modelo identifica corretamente 79% dos exemplos positivos reais da classe 1. No entanto, para a classe 0, a sensibilidade é alta, indicando que 87% dos exemplos positivos da classe 0 são capturados pelo modelo.

O *F1-Score* é uma métrica que combina precisão e sensibilidade em uma única medida, fornecendo uma média harmônica ponderada dessas duas métricas. Para a classe 0, o *F1-Score* é de 0,79, enquanto para a classe 1, é de 0,84, considerando que quanto mais próximo de 1, melhor o desempenho do modelo. O *F1-Score* é alto para a classe 1, refletindo um bom equilíbrio entre precisão e sensibilidade. Para a classe 0, o *F1-Score* é mais baixo, sugerindo um compromisso entre precisão e sensibilidade.

Por fim, a acurácia, é uma métrica que avalia a proporção de exemplos classificados corretamente em relação ao total de exemplos. No geral, o modelo alcança uma acurácia de 0,82, o que indica uma capacidade geral de fazer previsões corretas de 82% em toda a classificação considerando um total de 7.636 instâncias no conjunto de dados de teste, veja a Tabela 9 com os resultados.

Table 9: Métricas de Avaliação de Desempenho do modelo *Multilayer Perceptron*.

Classe	Precisão	Recall	F1-Score	Suporte
anã (0)	0.73	0.87	0.79	3029
gigante (1)	0.90	0.79	0.84	4607
Acurácia	-	-	0.82	7636

Resultados do modelo KNN

Análise de resultados do modelo *K-nearest neighbors*

As métricas obtidas para a avaliação desse algoritmo são apresentadas na Tabela 9. A classe 0 representa as estrelas

anãs e a classe 1 representa as estrelas gigantes. O modelo classificou incorretamente $500 + 1.073 = 1.573$ registros e classificou corretamente $2.529 + 3.534 = 6.063$ registros, respectivamente.

A classificação dos objetos pertencentes a classe 1 foi satisfatória (Taxa de VP = 87% aproximadamente), entretanto a classificação daqueles objetos pertencentes a classe 0 não apresentou um desempenho semelhante (Taxa de VP = 69% aproximadamente).

Resultados das Métricas de Avaliação de Desempenho do método *K-nearest neighbors*

A precisão avalia a proporção de previsões corretas em relação ao total de previsões para cada classe. No caso da classe 0, foi obtido uma precisão de 0,69, o que equivale a 69% das previsões corretas. Para a classe 1, a precisão é de 0,87, representando 87% de previsões corretas, concluindo que para a classe 1 é alta, indicando que quando o modelo prevê a classe 1, está correto 87% das vezes. No entanto, para a classe 0, a precisão é mais baixa, sugerindo um número significativo de falsos positivos.

A sensibilidade, ou *recall*, é outra métrica de avaliação que quantifica a proporção de exemplos positivos reais corretamente identificados pelo modelo em relação ao total de exemplos reais positivos. Para a classe 0, a sensibilidade é de 0,83, ou seja, 83% dos exemplos positivos foram capturados pelo modelo. Na classe 1, a sensibilidade é de 0,76, indicando que 76% dos exemplos positivos foram corretamente identificados, concluindo que sensibilidade para a classe 1 é razoável, significando que o modelo identifica corretamente 76% dos exemplos positivos reais da classe 1. No entanto, para a classe 0, a sensibilidade é alta, indicando que 83% dos exemplos positivos da classe 0 são capturados pelo modelo.

O *F1-Score* é uma métrica que combina precisão e sensibilidade em uma única medida, fornecendo uma média harmônica ponderada dessas duas métricas. Para a classe 0, o *F1-Score* é de 0,75, enquanto para a classe 1, é de 0,81, considerando que quanto mais próximo de 1, melhor o desempenho do modelo. O *F1-Score* é alto para a classe 1, refletindo um bom equilíbrio entre precisão e sensibilidade. Para a classe 0, o *F1-Score* é mais baixo, sugerindo um compromisso entre precisão e sensibilidade.

Por fim, a acurácia é uma métrica que avalia a proporção de exemplos classificados corretamente em relação ao total de exemplos. No geral, o modelo alcança uma acurácia de 0,79, o que indica uma capacidade geral de fazer previsões corretas de 79% em toda a classificação considerando um total de 7.636 instâncias no conjunto de dados de teste, veja a Tabela 10 com os resultados.

Table 10: Métricas de Avaliação de Desempenho do modelo *K-nearest neighbors*

Classe	Precisão	Recall	F1-Score	Suporte
anã (0)	0.69	0.83	0.75	3029
gigante (1)	0.87	0.76	0.81	4607
Acurácia	-	-	0.79	7636

6 VALIDAÇÃO E COMPARAÇÃO DOS RESULTADOS

Validação Cruzada

Para validar os resultados apresentados uma amostragem com validação cruzada foi utilizada tanto na obtenção dos melhores hiperparâmetros de cada modelo, quanto para validar os resultados do treinamento (validação cruzada aninhada). Para a validação cruzada o método *StratifiedKfold* da biblioteca *sci-kit-learn* foi utilizado. Esse método foi selecionado para manter uma distribuição adequada (visto o desbalanceamento da base) dos dados em *k* dobras (amostras) para os testes da validação cruzada. As amostras utilizadas na validação podem ser vistas na Figura 4 que apresenta os conjuntos de treino e teste utilizados para cada uma das 5 (*k* = 5) amostras utilizadas.

A Tabela 11 apresenta detalhadamente os resultados de cada modelo na validação cruzada.

Table 11: Resultados da validação cruzada para cada modelo treinado.

Modelo	Média	Desvio Padrão	Acurácia
MLP	0.826	0.0043	0.821
KNN	0.792	0.0064	0.793
XGB	0.821	0.0094	0.824
RF	0.821	0.0043	0.818
SVC	0.819	0.0053	0.815
BAG	0.815	0.0155	0.816

Para realizar a comparação da eficiência dos modelos considere a Tabela 12 e a curva ROC apresentada na Figura 5. Essa tabela e a imagem apresentam os resultados do treinamento de cada modelo após a validação cruzada, ademais observe a Tabela 13 com os hiperparâmetros utilizados em cada modelo.

A comparação estatística dos modelos foi realizada utilizando o teste t de *Student*, com um nível de significância de $\alpha = 0.05$. Esta seção detalha a significância, os intervalos de

Figure 4: Distribuição dos dados de treino e teste em cada uma das 5 partições utilizadas na validação cruzada. A linha com rótulo *class* apresenta a distribuição das classes Anã (ciano) e Gigante (marrom).

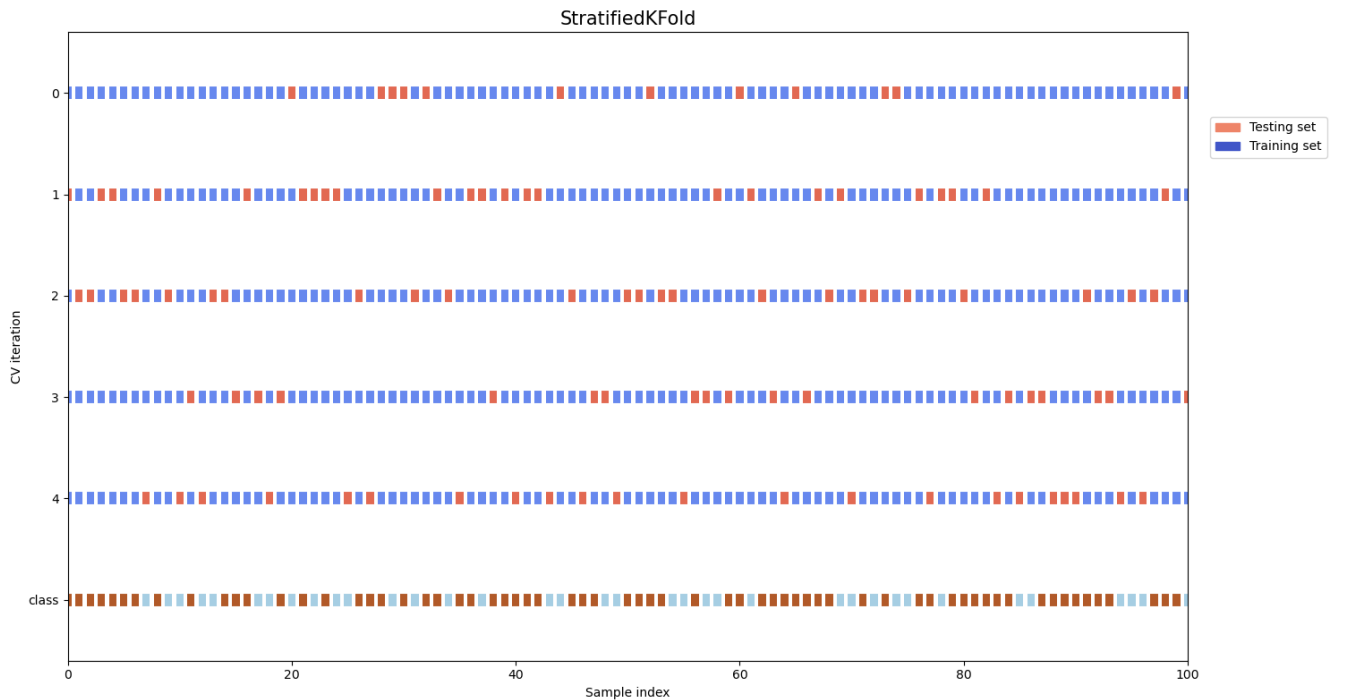


Table 12: Tabela de comparação entre os modelos utilizados.

Modelo	Classe	Precisão	Recall	F1-Score	Acurácia
Random Forest	anã (0)	0.71	0.90	0.79	0.82
	gigante (1)	0.92	0.76	0.83	
Bagging	anã (0)	0.73	0.84	0.78	0.81
	gigante (1)	0.89	0.79	0.84	
XGBoost	anã (0)	0.70	0.85	0.77	0.79
	gigante (1)	0.89	0.77	0.82	
SVM	anã (0)	0.70	0.91	0.79	0.87
	gigante (1)	0.93	0.74	0.83	
MLP	anã (0)	0.73	0.87	0.80	0.82
	gigante (1)	0.90	0.79	0.84	
KNN	anã (0)	0.70	0.83	0.76	0.79
	gigante (1)	0.87	0.76	0.81	

confiança, as médias e as estatísticas *t* para cada comparação entre modelos.

Teste *t* de Student
O valor crítico *t* (*T-Critical*) para todas as comparações foi de 2.306. Este valor é utilizado para determinar se a diferença entre as médias é estatisticamente significativa.

Table 13: Tabela de apresentação dos hiperparâmetros selecionados para os modelos através do algoritmos *GridSearchCV* ou *RandomSearchCV*

Modelo	Hiperparâmetros	Algoritmo Utilizado
MLP	$\{max_iter : 1000, hidden_layer_sizes : (11,), alpha : 0.0001, activation : logistic\}$	<i>RandomSearchCV</i>
KNN	$\{metric : euclidean, n_neighbors : 11, weights : uniform\}$	<i>GridSearchCV</i>
XGB	$\{n_estimators : 200, max_depth : 3, learning_rate : 0.3, gamma : 0\}$	<i>RandomSearchCV</i>
Random Forest	$\{criterion : entropy, max_depth : 5, max_features : sqrt, n_estimators : 100\}$	<i>GridSearchCV</i>
SVM	$\{kernel : rbf, gamma : 2, C : 0.1\}$	<i>RandomSearchCV</i>
Bagging	$\{n_estimators : 250, max_samples : 0.1, max_features : 1.0\}$	<i>RandomSearchCV</i>

Na comparação MLP vs KNN, o valor t foi de 17.043, indicando uma diferença significativa, com um valor- p de $1.43 * 10^{-7}$. O intervalo de confiança para o MLP foi de (0.825, 0.831) e para o KNN foi de (0.791, 0.799).

Para MLP vs RF, o valor t foi de 3.131, mostrando diferença significativa com um valor- p de 0.0140. Os intervalos de confiança foram (0.825, 0.831) para MLP e (0.817, 0.826) para RF.

Em MLP vs SVC, o valor t foi de 4.971, significativo com um valor- p de 0.0011. Os intervalos de confiança foram (0.825, 0.831) para MLP e (0.817, 0.823) para SVC.

Na comparação MLP vs BAG, o valor t foi de 2.291, não sendo significativo com um valor- p de 0.0512. Os intervalos de confiança foram (0.825, 0.831) para MLP e (0.807, 0.829) para BAG.

Entre KNN vs RF, o valor t foi de -11.692, indicando uma diferença significativa com um valor- p de $2.61 * 10^{-6}$. Os intervalos de confiança foram (0.791, 0.799) para KNN e (0.817, 0.826) para RF.

Na comparação KNN vs SVC, o valor t foi de -13.082, mostrando uma diferença significativa com um valor- p de $1.11 * 10^{-6}$. Os intervalos de confiança foram (0.791, 0.799) para KNN e (0.817, 0.823) para SVC.

Para KNN vs BAG, o valor t foi de -5.379, significativo com um valor- p de 0.00066. Os intervalos de confiança foram (0.791, 0.799) para KNN e (0.807, 0.829) para BAG.

Entre RF vs SVC, o valor t foi de 0.738, não indicando diferença significativa com um valor- p de 0.4816. Os intervalos de confiança foram (0.817, 0.826) para RF e (0.817, 0.823) para SVC.

Na comparação RF vs BAG, o valor t foi de 0.793, sem diferença significativa e um valor- p de 0.4509. Os intervalos de confiança foram (0.817, 0.826) para RF e (0.807, 0.829) para BAG.

Para SVC vs BAG, o valor t foi de 0.480, também sem diferença significativa e um valor- p de 0.6442. Os intervalos

de confiança foram (0.817, 0.823) para SVC e (0.807, 0.829) para BAG.

Discussão dos resultados da validação cruzada e do teste t

Na análise dos resultados obtidos pela validação cruzada e teste, observamos que cada modelo apresentou características distintas em termos de desempenho. O modelo MLP destacou-se com uma média de 0.828 na validação cruzada, acompanhada de um desvio padrão de 0.0021, o que reflete uma alta consistência em seu desempenho. Este resultado foi corroborado no teste, onde o MLP alcançou uma acurácia de 0.824, reafirmando sua eficácia.

Em contraste, o modelo KNN registrou uma média mais baixa na validação cruzada, de 0.795, com um desvio padrão de 0.0032, indicando um desempenho moderado. Esta tendência foi confirmada no teste, onde o KNN atingiu uma acurácia de 0.790, sugerindo que, apesar de ser um modelo mais simples, pode não ser tão eficaz quanto os outros para este conjunto específico de dados.

O modelo XGB, por sua vez, mostrou uma boa performance com uma média de 0.798 na validação cruzada e um desvio padrão de 0.0076, revelando um desempenho robusto. No teste, a acurácia de 0.799 demonstrou a eficácia consistente do modelo.

Da mesma forma, o modelo RF exibiu um desempenho estável com uma média de 0.822 na validação cruzada e um desvio padrão de 0.0033. No teste, a acurácia foi de 0.813, alinhando-se com os resultados obtidos na validação cruzada.

O modelo SVC também apresentou resultados competitivos, com uma média de 0.820 e um desvio padrão de 0.0022 na validação cruzada. Este desempenho foi reiterado no teste, onde o SVC alcançou uma acurácia de 0.810, demonstrando uma consistência entre a validação e o teste.

Por fim, o modelo BAG registrou uma média de 0.818 na validação cruzada, com um desvio padrão de 0.0080, indicando uma variabilidade maior em seu desempenho. No teste, a acurácia foi de 0.814, o que está em consonância com os resultados da validação cruzada.

A análise geral destes resultados mostra que, enquanto o MLP emerge como o modelo mais eficaz em termos de acurácia média, os modelos RF, SVC e XGB também demonstraram ser opções sólidas e consistentes. O modelo KNN, embora tenha apresentado um desempenho inferior em comparação com os outros, pode ser valioso em situações onde a simplicidade e a facilidade de interpretação são prioritárias. O modelo BAG, com sua maior variabilidade, pode ser adequado em contextos onde a diversidade nas abordagens de modelagem é benéfica.

7 CONCLUSÃO

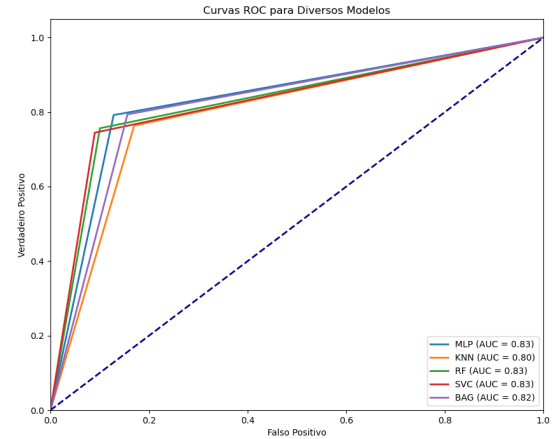
A avaliação das Curvas ROC, que ilustram a relação entre a taxa de verdadeiros positivos (TPR - *True Positive Rate*) e a taxa de falsos positivos (FPR - *False Positive Rate*), revela insights importantes sobre o desempenho dos modelos analisados. Embora o modelo de *Bagging* demonstre uma proximidade marginalmente maior ao ponto ideal no canto superior esquerdo do gráfico, conforme ilustrado na Figura 5, seu desempenho é ligeiramente ultrapassado pelos modelos *MLP*, *Random Forest* e *SVC*, os quais apresentam uma AUC de 0.83, indicando um equilíbrio otimizado entre TPR e FPR. O modelo *Bagging* segue de perto com uma AUC de 0.82, sugerindo ainda um desempenho robusto.

Interessante notar que os modelos baseados em árvores de decisão, especificamente *Random Forest* e *Bagging*, compartilham áreas sob a curva ROC similares, refletindo uma eficácia comparável no contexto da classificação. O modelo *KNN*, embora apresente uma AUC ligeiramente menor de 0.80, ainda exibe uma capacidade considerável de discriminação entre as classes.

Contrariamente ao exposto no texto original, o modelo *SVM* não só iguala mas também se alinha aos modelos *Random Forest* e *Bagging* em termos de AUC, todos com 0.83, contrapondo a afirmação anterior de uma AUC de 0.72 para o *SVM*. Esses resultados destacam a competitividade do *SVM* frente aos modelos baseados em árvores de decisão e enfatizam a precisão de sua classificação.

Portanto, com base nas curvas ROC e nas respectivas áreas sob a curva, podemos concluir que, embora existam nuances em seus desempenhos, todos os modelos demonstraram uma capacidade notável de classificação, com o *MLP*, *Random Forest*, *SVC* e *Bagging* apresentando desempenhos quase equivalentes e o *KNN* não ficando muito atrás.

Figure 5: Gráfico de comparação entre os métodos *Random Forest*, *Bagging*, *XGBoost* e *SVM*.



8 CÓDIGO DESENVOLVIDO

Para esse trabalho o código com a validação cruzada e com os testes pode ser obtido no github através do acesso ao repositório: *Stellar-Classification*.

REFERENCES

- [1] F. Arenou and X. Luri. 1998. Distances and absolute magnitudes from trigonometric parallaxes. , 20 pages. <https://doi.org/10.48550/arXiv.astro-ph/9812094> arXiv:astro-ph/astro-ph/9812094
- [2] Martin Aumüller, Sarel Har-Peled, Sepideh Mahabadi, Rasmus Pagh, and Francesco Silvestri. 2022. Sampling a Near Neighbor in High Dimensions — Who is the Fairest of Them All? *ACM Trans. Database Syst.* 47, 1, Article 4 (apr 2022), 40 pages. <https://doi.org/10.1145/3502867>
- [3] Martin Aumüller, Sarel Har-Peled, Sepideh Mahabadi, Rasmus Pagh, and Francesco Silvestri. 2022. Sampling near Neighbors in Search for Fairness. *Commun. ACM* 65, 8 (jul 2022), 83–90. <https://doi.org/10.1145/3543667>
- [4] Gustavo E. A. P. A. Batista, Ronaldo C. Prati, and Maria Carolina Monard. 2004. A Study of the Behavior of Several Methods for Balancing Machine Learning Training Data. *SIGKDD Explor. Newsl.* 6, 1 (jun 2004), 20–29. <https://doi.org/10.1145/1007730.1007735>
- [5] John J. Bochanski, Suzanne L. Hawley, Kevin R. Covey, Andrew A. West, I. Neill Reid, David A. Golimowski, and Željko Ivezić. 2010. THE LUMINOSITY AND MASS FUNCTIONS OF LOW-MASS STARS IN THE GALACTIC DISK. II. THE FIELD. *The Astronomical Journal* 139, 6 (may 2010), 2679. <https://doi.org/10.1088/0004-6256/139/6/2679>
- [6] Jair Cervantes, Farid Garcia-Lamont, Lisbeth Rodríguez-Mazahua, and Asdrubal Lopez. 2020. A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing* 408 (2020), 189–215. <https://doi.org/10.1016/j.neucom.2019.10.118>
- [7] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic Minority over-Sampling Technique. *J. Artif. Int. Res.* 16, 1 (jun 2002), 321–357.
- [8] Soledad Galli. 2023. Under-sampling. In *imbalanced-learn*. imbalanced-learn. https://github.com/scikit-learn-contrib/imbalanced-learn/blob/master/doc/under_sampling.rst
- [9] Sunetra Giridhar. 2010. Advances in Spectral Classification. arXiv:astro-ph.SR/1004.1294
- [10] Hanghang Tong Jiawei Han, Jian Pei. 2022. *Data Mining: Concepts and Techniques* (fourth edition ed.). Elsevier, 50 Hampshire Street, 5th Floor, Cambridge, MA 02139, United States.
- [11] W W Morgan, Philip C Keenan, Edith Kellman. 2004. An Atlas of Stellar Spectra with an Outline of Spectral Classification. , 44 pages.
- [12] V. Khramtsov, I. B. Vavilova, D. V. Dobrycheva, M. Yu. Vasylenko, O. V. Melnyk, A. A. Elyiv, V. S. Akhmetov, and A. M. Dmytrenko. 2022. Machine learning technique for morphological classification of galaxies from the SDSS. III. Image-based inference of detailed features. *Kosmichna Nauka i Tekhnologiya* 28, 5 (sep 2022), 27–55. <https://doi.org/10.15407/knit2022.05.027>
- [13] Kiran Maharana, Surajit Mondal, and Bhushankumar Nemade. 2022. A review: Data pre-processing and data augmentation techniques. *Global Transitions Proceedings* 3, 1 (2022), 91–99. <https://doi.org/10.1016/j.gltp.2022.04.020> International Conference on Intelligent Engineering Approach(ICIEA-2022).
- [14] Ibomoiye Domor Mienye and Yanxia Sun. 2022. A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects. *IEEE Access* 10 (2022), 99129–99149. <https://doi.org/10.1109/ACCESS.2022.3207287>
- [15] Fionn Murtagh. 1991. Multilayer perceptrons for classification and regression. *Neurocomputing* 2, 5 (1991), 183–197. [https://doi.org/10.1016/0925-2312\(91\)90023-5](https://doi.org/10.1016/0925-2312(91)90023-5)
- [16] Olcay Plevne, Özgecan Önal Taş, Selçuk Bilir, and George M. Seabroke. 2020. Multiwavelength Absolute Magnitudes and Colors of Red Clump Stars in the Gaia Era. *The Astrophysical Journal* 893, 2 (apr 2020), 108. <https://doi.org/10.3847/1538-4357/ab80bb>
- [17] Benjamin R. Roulston, Paul J. Green, and Aurora Y. Kesseli. 2020. Classifying Single Stars and Spectroscopic Binaries Using Optical Stellar Templates. *The Astrophysical Journal Supplement Series* 249, 2 (aug 2020), 34. <https://doi.org/10.3847/1538-4365/aba1e7>
- [18] Stuart J Russell. 2021. *Artificial intelligence : a modern approach* (fourth edition ed.). Pearson, 221 River Street, Hoboken, United States.
- [19] Kaushal Sharma, Ajit Kembhavi, Aniruddha Kembhavi, T Sivaran, Sheelu Abraham, and Kaustubh Vaghmare. 2019. Application of convolutional neural networks for stellar spectral classification. *Monthly Notices of the Royal Astronomical Society* 491, 2 (11 2019), 2280–2300. <https://doi.org/10.1093/mnras/stz3100> arXiv:https://academic.oup.com/mnras/article-pdf/491/2/2280/31194753/stz3100.pdf
- [20] Dalwinder Singh and Birmohan Singh. 2020. Investigating the impact of data normalization on classification performance. *Applied Soft Computing* 97 (2020), 105524. <https://doi.org/10.1016/j.asoc.2019.105524>
- [21] Jingwen Wang, Xuyang Jing, Zheng Yan, Yulong Fu, Witold Pedrycz, and Laurence T. Yang. 2020. A Survey on Trust Evaluation Based on Machine Learning. *ACM Comput. Surv.* 53, 5, Article 107 (sep 2020), 36 pages. <https://doi.org/10.1145/3408292>
- [22] Bowen Xu, Amirreza Shirani, David Lo, and Mohammad Amin Alipour. 2018. Prediction of Relatedness in Stack Overflow: Deep Learning vs. SVM: A Reproducibility Study. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '18)*. Association for Computing Machinery, New York, NY, USA, Article 21, 10 pages. <https://doi.org/10.1145/3239235.3240503>
- [23] Jiawei Yang, Susanto Rahardja, and Pasi Fränti. 2019. Outlier Detection: How to Threshold Outlier Scores?. In *Proceedings of the International Conference on Artificial Intelligence, Information Processing and Cloud Computing (AIIPCC '19)*. Association for Computing Machinery, New York, NY, USA, Article 37, 6 pages. <https://doi.org/10.1145/3371425.3371427>